```python
import os
import requests
from bs4 import BeautifulSoup
from datetime import datetime, timedelta
import google.generativeai as genai
import json
import time

# --- 設定 ---
BLOG_LIST_URL = "https://www.hinatazaka46.com/s/official/diary/member/list"
BLOG_BASE_URL = "https://www.hinatazaka46.com"
LAST_FETCHED_FILE = "last_fetched.txt"
SLACK_WEBHOOK_URL = os.environ["SLACK_WEBHOOK_URL"]
GEMINI_API_KEY = os.environ["GEMINI_API_KEY"]
BUFFER_MINUTES = 10  # 取りこぼし防止バッファ

# --- Gemini初期化 ---
genai.configure(api_key=GEMINI_API_KEY)
model = genai.GenerativeModel("gemini-1.5-pro")


def load_last_fetched() -> datetime:
    """前回実行時刻を読み込む。なければ24時間前を返す。"""
    if os.path.exists(LAST_FETCHED_FILE):
        with open(LAST_FETCHED_FILE, "r") as f:
            dt_str = f.read().strip()
            return datetime.fromisoformat(dt_str)
    else:
        return datetime.now() - timedelta(hours=24)


def save_last_fetched(dt: datetime):
    """現在時刻を保存。"""
    with open(LAST_FETCHED_FILE, "w") as f:
        f.write(dt.isoformat())


def fetch_blog_list() -> list[dict]:
    """ブログ一覧ページから記事リストを取得。"""
    headers = {"User-Agent": "Mozilla/5.0"}
    resp = requests.get(BLOG_LIST_URL, headers=headers, timeout=15)
    resp.raise_for_status()

    soup = BeautifulSoup(resp.text, "html.parser")
```

```
    articles = soup.select("div.p-blog-group .p-blog-article")

    results = []
    for article in articles:
        title_el = article.select_one(".c-blog-article__title")
        name_el = article.select_one(".c-blog-article__name")
        date_el = article.select_one(".c-blog-article__date")
        link_el = article.select_one(".c-button-blog-detail")

        if not all([title_el, name_el, date_el, link_el]):
            continue

        date_str = date_el.get_text(strip=True)
        # 例: "2025.04.09 18:30"
        try:
            pub_date = datetime.strptime(date_str, "%Y.%m.%d %H:%M")
        except ValueError:
            continue

        href = link_el.get("href", "")
        full_url = BLOG_BASE_URL + href if href.startswith("/") else href

        results.append({
            "title": title_el.get_text(strip=True),
            "member": name_el.get_text(strip=True),
            "date": pub_date,
            "url": full_url,
        })

    return results


def fetch_blog_content(url: str) -> str:
    """個別ブログページから本文を取得。"""
    headers = {"User-Agent": "Mozilla/5.0"}
    resp = requests.get(url, headers=headers, timeout=15)
    resp.raise_for_status()

    soup = BeautifulSoup(resp.text, "html.parser")
    content_el = soup.select_one(".c-blog-article__text")
    if content_el:
        return content_el.get_text(separator="\n", strip=True)
    return ""


def summarize(member: str, title: str, content: str) -> str:
    """Geminiでブログを要約。"""
```

```
    prompt = f"""以下は日向坂46のメンバー「{member}」のブログです。
タイトル: {title}

本文:
{content[:3000]}

---
このブログを日本語で3〜5行に要約してください。メンバーの気持ちや出来事を中心にまとめてください。"

    response = model.generate_content(prompt)
    return response.text.strip()


def post_to_slack(member: str, title: str, url: str, summary: str, pub_date: date
    """Slackにメッセージを投稿。"""
    date_str = pub_date.strftime("%Y/%m/%d %H:%M")
    message = {
        "text": f"*📝 {member}* がブログを更新しました（{date_str}）\n*{title}*\n{url
        "attachments": [
            {
                "color": "#f5a623",
                "text": summary,
            }
        ],
    }
    resp = requests.post(SLACK_WEBHOOK_URL, json=message, timeout=10)
    resp.raise_for_status()


def main():
    now = datetime.now()
    last_fetched = load_last_fetched()
    threshold = last_fetched - timedelta(minutes=BUFFER_MINUTES)

    print(f"取得対象: {threshold.isoformat()} 以降のブログ")

    articles = fetch_blog_list()
    new_articles = [a for a in articles if a["date"] >= threshold]

    print(f"新着: {len(new_articles)}件")

    for article in new_articles:
        print(f"処理中: {article['member']} - {article['title']}")
        try:
            content = fetch_blog_content(article["url"])
            if not content:
                print("  本文取得失敗、スキップ")
```

```
                continue

            summary = summarize(article["member"], article["title"], content)
            post_to_slack(
                article["member"],
                article["title"],
                article["url"],
                summary,
                article["date"],
            )
            print(f"  Slack送信完了")
            time.sleep(2)  # API負荷軽減

        except Exception as e:
            print(f"  エラー: {e}")
            continue

    save_last_fetched(now)
    print("完了")


if __name__ == "__main__":
    main()
```